

CHƯƠNG 2. PHÂN TÍCH VÀ THIẾT KẾ LỚP

2.1. Phân tích bài toán

- Trong bối cảnh chuyển đổi số mạnh mẽ hiện nay, các doanh nghiệp bán lẻ, siêu thị đang đối mặt với nhu cầu cấp thiết trong việc tự động hóa quy trình quản lý bán hàng, nhằm giảm thiểu sai sót, tối ưu hóa chi phí vận hành và nâng cao trải nghiệm khách hàng.
- Từ thực tiễn đó, nhóm đã xây dựng một hệ thống phần mềm quản lý bán hàng tổng thể dựa trên kỹ thuật lập trình hướng đối tượng (OOP), tận dụng các nguyên lý thiết kế và mô hình hóa đối tượng để đảm bảo tính mở rộng, tái sử dụng và bảo trì cao. Dự án hướng đến việc mô phỏng một hệ thống bán hàng có đầy đủ quy trình nghiệp vụ từ quản lý sản phẩm, khách hàng, đơn hàng cho đến hóa đơn, thanh toán và nhân viên.
- Hệ thống bán hàng siêu thị sẽ đáp ứng những nhu cầu của cửa hàng:
 - + Quản lý thông tin sản phẩm và thông tin khách hàng
 - + Thống kê doanh thu của cửa hàng
 - + Quản lý thông tin người quản lý của mỗi cửa hàng và nhân viên bán hàng trong cửa hàng đó
 - + Tạo và quản lý đơn hàng
 - + Thanh toán và xuất hóa đơn
 - + Thống kê doanh sách đơn hàng và hóa đơn

2.2. Thiết kế lớp và Sơ đồ lớp (Class diagrams)

2.2.1. Thiết kế lớp

- Qua quá trình khảo sát nghiệp vụ thực tế, nhóm đưa ra các chức năng chính mà hệ thống phải có như sau:
 - + Quản lý sản phẩm (Product): thêm, sửa, xóa sản phẩm; phân loại theo nhóm (thực phẩm, đồ uống, điện tử, may mặc, gia dụng...).
 - + Quản lý khách hàng (Customer): lưu trữ thông tin khách hàng, lịch sử mua hàng, chính sách ưu đãi (khuyến mãi theo từng loại khách hàng).
 - + Quản lý đơn hàng (Order): tạo đơn hàng, cập nhật chi tiết từng mặt hàng, tính tổng tiền và thành tiền dựa trên khuyến mãi của từng loại khách hàng, quản lý lịch sử đơn hàng.
 - + Quản lý hóa đơn (Invoice): tạo, in hóa đơn thanh toán và quản lý lịch sử giao dịch.
 - + Quản lý nhân viên (Employee): quản lý thông tin, phân quyền truy cập cho nhân viên (quản lý, thu ngân).
 - + Quản lý thanh toán (Payments): quản lý các phương thức thanh toán như tiền mặt, quét mã QR hay thẻ ngân hàng.

+ Báo cáo và thống kê: doanh thu, số lượng đơn hàng và khách hàng.

→ Dựa vào các phân tích ở trên, chương trình được thiết kế theo mô hình hướng đối tượng, trong đó các thực thể (entity) chính được mô hình hóa thành các lớp như sau:

+ Lớp Product (Sản phẩm): Thể hiện thông tin về sản phẩm trong cửa hàng như mã sản phẩm, tên, loại hàng, số lượng tồn kho, giá bán, đơn vị tính và các thuộc tính đặc trưng khác. Đây là lớp trung tâm của hệ thống vì hầu hết các chức năng như bán hàng, nhập kho, hóa đơn hay đơn hàng đều có liên quan trực tiếp đến sản phẩm.

+ Lớp Customer (Khách hàng): Lưu trữ thông tin khách hàng gồm mã khách hàng, họ tên, số điện thoại, địa chỉ, loại khách hàng và lịch sử mua hàng. Lớp này giúp hệ thống quản lý và tra cứu thông tin khách hàng, hỗ trợ chăm sóc khách hàng và phân tích hành vi mua sắm.

+ Lớp Employee (Nhân viên): Đại diện cho nhân viên hệ thống, lưu trữ thông tin như mã nhân viên, họ tên, chức vụ, lương. Lớp này có thể được thừa kế hoặc mở rộng cho các vai trò khác nhau như quản lý, thu ngân.

+ Lớp Order (Đơn hàng): Chứa thông tin về các đơn hàng được tạo ra từ phía khách hàng hoặc nhân viên bán hàng, gồm mã đơn, khách hàng, nhân viên bán hàng, ngày lập, tổng tiền và trạng thái đơn hàng. Lớp này đóng vai trò cầu nối giữa khách hàng và sản phẩm.

+ Lớp OrderDetails (Chi tiết đơn hàng): Liên kết giữa Order và Product, lưu chi tiết từng sản phẩm trong một đơn hàng như mã sản phẩm, số lượng, đơn giá, tổng tiền và thành tiền. Lớp này giúp hệ thống theo dõi chi tiết quá trình bán hàng.

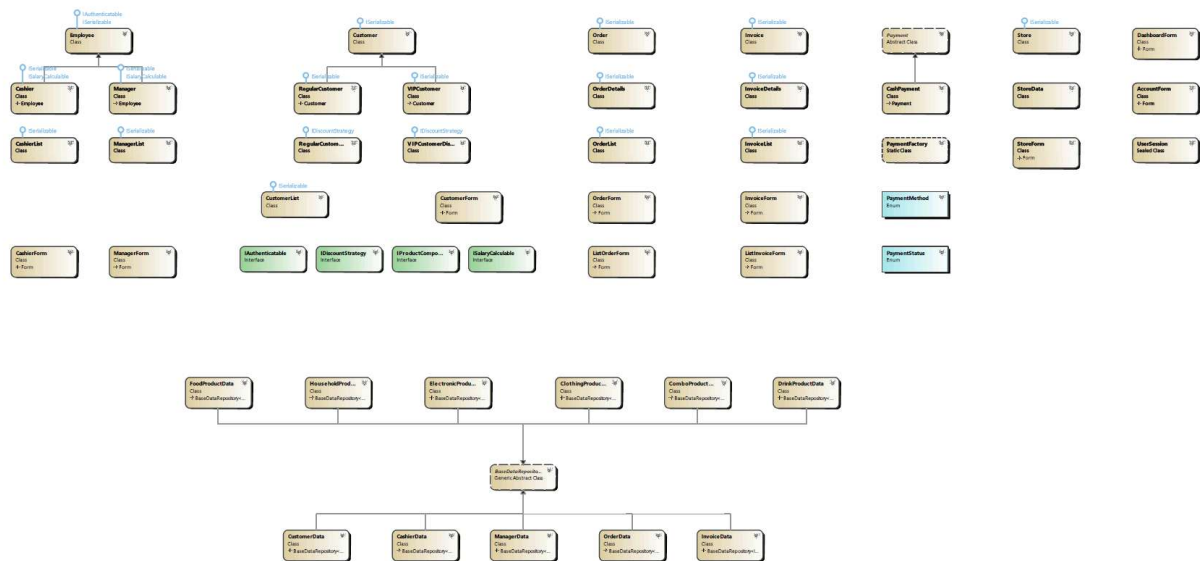
+ Lớp Invoice (Hóa đơn): Đại diện cho chứng từ thanh toán, lưu thông tin về ngày lập hóa đơn, mã khách hàng, nhân viên lập, tổng tiền và thành tiền.

+ Lớp InvoiceDetails (Chi tiết hóa đơn): Mô tả chi tiết từng sản phẩm trong hóa đơn, bao gồm số lượng, đơn giá, khuyến mãi, tổng tiền và thành tiền. Lớp này tương tự như OrderDetails nhưng phục vụ riêng cho quá trình thanh toán.

+ Lớp Payments (Thanh toán): Hệ thống hỗ trợ thanh toán chi tiết với các phương thức đa dạng như tiền mặt, QR hay thẻ ngân hàng, ở chức năng tiền mặt còn hỗ trợ tính toán tự động số tiền khách và cần phải thối giúp hỗ trợ tối đa nhân viên bán hàng

+ Lớp System Manager Quản lý hệ thống: Là lớp trung tâm điều phối, chịu trách nhiệm quản lý dữ liệu, điều hướng hoạt động của các lớp như thêm, sửa, xóa, tìm kiếm, tính toán doanh thu và thống kê. Lớp này giúp đảm bảo tính nhất quán và logic của chương trình.

2.2.2. Sơ đồ lớp



Hình 2.1. Sơ đồ lớp

Link ảnh trong Drive: [PDF Diagram-OOP.pdf](#)

2.3. Cài đặt các lớp chức năng

2.3.1. Tổng quan mô hình lớp

a. Nhóm sản phẩm (Product)

- Quản lý toàn bộ thông tin (như mã sản phẩm, tên sản phẩm, giá, số lượng) liên quan đến các loại sản phẩm được bán trong cửa hàng.

- Lớp Product: lớp trừu tượng (abstract class) - đại diện cho mọi sản phẩm.

- Kế thừa các interface:

- ISerializable: cho phép lưu trữ và đọc dữ liệu sản phẩm.
- IProductComponent: dùng trong Composite Pattern (quản lý nhóm sản phẩm).

- Các lớp kế thừa: FoodProduct, DrinkProduct, ClothingProduct, ElectronicProduct, HouseholdProduct và ComboProduct. Mỗi lớp thể hiện một loại sản phẩm cụ thể, mỗi loại mở rộng thêm các thuộc tính đặc thù riêng như

- FoodProduct thêm thuộc tính ExpirationDate để quản lý hạn sử dụng của sản phẩm
- DrinkProduct thêm thuộc tính Carbonated để xác định nước uống có ga hay không
- ClothingProduct thêm thuộc tính Size để quản lý kích cỡ sản phẩm
- ElectronicProduct thêm thuộc tính WarrantyPeriod để quản lý chính sách thời gian bảo hành
- HouseholdProduct thêm thuộc tính Brand để phân biệt sản phẩm theo thương hiệu.

- Lớp ComboProduct: Cho phép gộp nhiều sản phẩm lại thành một combo (Đây là ứng dụng của Composite Design Pattern)

- Các lớp hỗ trợ:

- Các lớp: FoodProductList, DrinkProductList, ClothingProductList, ElectronicProductList, HouseholdProductList được dùng để quản lý danh sách sản phẩm
- Các lớp: FoodProductData, DrinkProductData, ClothingProductData, ElectronicProductData, HouseholdProductData được dùng để quản lý dữ liệu sản phẩm
- Các lớp: FoodProductForm, DrinkProductForm, ClothingProductForm, ElectronicProductForm, HouseholdProductForm. Đây là giao diện hiển thị sản phẩm

→ Các chức năng chính: Thêm, sửa, xóa, lưu trữ sản phẩm. Tính tổng giá trị và hiển thị thông tin sản phẩm. Cho phép tạo nhóm sản phẩm (combo).

b. Nhóm lớp Nhân Viên (Employee)

- Quản lý nhân sự trong siêu thị: nhân viên thu ngân và quản lý.

- Lớp Employee: lớp cha, và kế thừa các interface: IAuthenticatable (xác định vai trò), ISerializable (cho phép lưu trữ và đọc dữ liệu nhân viên).

- Các lớp con: Manager (người quản lý, quản lý sản phẩm, nhân viên, đơn hàng và hóa đơn), Cashier (nhân viên thu ngân, thực hiện bán hàng và in hóa đơn), kế thừa ISalaryCalculable (tính tiền lương tổng dựa trên số giờ làm và lương cơ bản).

- Các lớp hỗ trợ: ManagerData, ManagerList, CashierData, CashierList (Quản lý dữ liệu, danh sách nhân viên quản lý và nhân viên bán hàng), ManagerForm, CashierForm (Quản lý thông tin của nhân viên quản lý và nhân viên bán hàng)

- MainFormAdmin, MainFormCashier (giao diện chính cho từng vai trò của nhân viên)
Các chức năng chính: Đăng nhập, xác nhận quyền, quản lý thông tin nhân viên, phân quyền theo vai trò của từng nhân viên.

c. Nhóm lớp Khách Hàng (Customer)

- Quản lý thông tin khách hàng, phân loại khách hàng và chính sách ưu đãi giảm giá.

- Lớp Customer: lớp cha kế thừa interface ISerializable (Cho phép lưu trữ và đọc dữ liệu khách hàng)

- Các lớp con: RegularCustomer (Khách hàng thường); VIPCustomer (Khách hàng VIP).

- Các lớp chiến lược (Strategy): RegularCustomerDiscountStrategy, VIPCustomerDiscountStrategy triển khai từ IDiscountStrategy (Mỗi lớp định nghĩa cách tính giảm giá riêng biệt cho từng loại khách hàng)

- Các lớp hỗ trợ: CustomerList, CustomerData, CustomerForm (lưu trữ và hiển thị thông tin khách hàng)

- Các chức năng chính: Quản lý hồ sơ khách hàng, phân loại khách hàng...

d. Nhóm lớp Đơn hàng - Hóa đơn (Order - Invoice)

- Xử lý các tác vụ mua hàng, lập đơn hàng, lập hóa đơn, quản lý các giao dịch.
- Các lớp chính: Order, Invoice, OrderDetails, InvoiceDetails, (lưu thông tin giao dịch)
- Các lớp dữ liệu: OrderData, InvoiceData (xử lý đọc/ghi dữ liệu)
- Các lớp danh sách: OrderList, InvoiceList (quản lý danh sách đơn hàng và hóa đơn)
- Các lớp giao diện: OrderForm, InvoiceForm, ListOrderForm, ListInvoiceForm (hiển thị và thi hành các chức năng của đơn hàng và hóa đơn)
- Các chức năng chính: Tạo, lưu và hiển thị đơn hàng, hóa đơn. Tính tổng giá trị đơn hàng, xuất hóa đơn và quản lý lịch sử giao dịch.

e. Nhóm lớp Thanh toán (Payments)

- Xử lý toàn bộ quá trình thanh toán, bao gồm tính toán tiền thừa (tiền mặt) và thực hiện thanh toán
- Các lớp chính: CashPayment, Payment, PaymentFactory, PaymentMethod, PaymentStatus, (CardPayment và QRPayment có thể mở rộng thêm).
- Các chức năng chính: tính toán và trả về số tiền thừa ngay lập tức, thực hiện thanh toán bằng các hình thức khác nhau (tiền mặt, mã QR,...), sinh và trả về mã số giao dịch cho mỗi lần thanh toán.

f. Nhóm quản lý hệ thống (System Manager)

- Cung cấp giao diện người dùng và các tác vụ tiện ích cho hệ thống
- Lớp Program: lớp chính khởi tạo chạy lớp ban đầu.
- Form MainInterface, SignIn, SignUp: Form giao diện chính, xử lý đăng nhập, tạo tài khoản và phân loại quyền đăng nhập theo từng vai trò (Admin/Cashier).
- Form DashboardForm: giao diện chính trong MainFormAdmin (báo cáo doanh thu, số lượng khách hàng, nhân viên và số lượng đơn hàng theo thời gian thực), AccountForm (Hiển thị thông tin tài khoản đăng nhập và các thông số liên quan)

2.3.2. Các Tính chất cơ bản của Lập trình hướng đối tượng (OOP)

a. Tính đóng gói (Encapsulation)

- Các lớp như Product, Customer và Employee đều có các thuộc tính được khai báo với phạm vi truy cập **private**, nhằm bảo vệ dữ liệu bên trong lớp.
- Việc truy cập và thay đổi giá trị của các thuộc tính này được kiểm soát thông qua các phương thức công khai (public) dưới dạng property { get; set; }.
- Ví dụ trong lớp Customer:

```

private string name;
public string Name { get { return name; }
    set
    {
        if (string.IsNullOrEmpty(value))
        {
            throw new ArgumentException("Tên khách hàng không được để trống hoặc
rỗng!");
        }
        name = value;
    }
}

```

b. Tính Trừu tượng (Abstraction)

- Các lớp interface như:

+ IProductComponent

- Mục đích: Định nghĩa cấu trúc chung cho Composite Pattern, cho phép sản phẩm đơn và sản phẩm nhóm (combo) được xử lý thống nhất. Giúp mô hình hóa các sản phẩm đơn và combo trong hệ thống siêu thị một cách linh hoạt và mở rộng.

- Các phương thức:

+ CalculateTotal(), CalculateDiscount(): Tính toán giá trị và giảm giá.

+ GetDisplayInfo(), GetShortInfo(): Lấy thông tin hiển thị.

+ IsComposite(): Kiểm tra xem đối tượng có phải là nhóm sản phẩm (composite) không.

+ GetChildren(): Lấy danh sách các thành phần con nếu là composite.

+ IDiscountStrategy

- Mục đích: Định nghĩa các chiến lược giảm giá khác nhau (Strategy Pattern). Giúp thay đổi linh hoạt các chính sách khuyến mãi (ví dụ: giảm theo phần trăm, giảm theo hóa đơn, giảm theo dạng khách hàng,...).

- Các phương thức:

+ CalculateDiscount(decimal totalAmount): Tính tiền giảm giá dựa trên tổng giá trị đơn hàng.

+ GetDiscountPercentage(): Lấy phần trăm giảm giá.

+ GetStrategyName(): Lấy tên chiến lược giảm giá.

+ GetDescription(): Mô tả chi tiết chiến lược giảm giá.

```

public interface IProductComponent
{
    string Id { get; set; }
    string Name { get; set; }
    decimal Price { get; set; }
    decimal Quantity { get; set; }
    decimal CalculateTotal();
    decimal CalculateDiscount(decimal discountPercentage);
    string GetDisplayInfo();
    string GetShortInfo();
    bool IsComposite();
    List<IProductComponent> GetChildren();
}

```

+ IAuthenticatable

- Mục đích: Định nghĩa hành vi trừu tượng liên quan đến xác thực người dùng trong hệ thống. Giúp chuẩn hóa cách xác thực và phân quyền người dùng, đảm bảo tính bảo mật và rõ ràng trong quản lý hệ thống.

- Các phương thức:

+ GetRole(): Lấy vai trò người dùng (quản lý, nhân viên, khách hàng,...).

+ ISalaryCalculable

- Mục đích: Định nghĩa hành vi trừu tượng liên quan đến tính toán lương cho nhân viên. Mặc định chương trình tính toán với số giờ làm hàng ngày là 8h, mức lương mỗi giờ tương ứng với vai trò của từng nhân viên.

- Các phương thức:

+ HourlyRate: Tiền lương cơ bản tương ứng cho mỗi nhân viên

+ Salary: Tính tiền lương tổng dựa trên lương cơ bản nhân với số giờ mặc định (8 tiếng)

c. Tính Kế thừa (Inheritance)

- Lớp Product đóng vai trò là lớp cha, các lớp con như FoodProduct, DrinkProduct, ClothingProduct, ElectronicProduct, HouseholdProduct kế thừa (inherit) từ lớp này, và mở rộng thêm các thuộc tính đặc trưng riêng của từng loại hàng hóa (như hạn sử dụng, thương hiệu, size, ...)

- Ví dụ lớp DrinkProduct kế thừa từ lớp Product:

```

public class DrinkProduct : Product, ISerializable { }

```

d. Tính Đa hình (Polymorphism)

- Từ khóa `abstract` được sử dụng trong lớp cha để khai báo một phương thức trừu tượng, cho phép các lớp con ghi đè (`override`) và định nghĩa hành vi riêng biệt cho cùng một phương thức.

- Ví dụ trong lớp `DrinkProduct`:

```
public abstract class Product
{
    public abstract string Info();
}
public class DrinkProduct : Product
{
    public override string Info()
    {
        return $"Có gas: {Carbonated}";
    }
}
```

→ Khi phương thức `Info()` được gọi thông qua đối tượng kiểu `Product`, chương trình sẽ tự động thực thi các câu lệnh của phương thức trong lớp con tương ứng.

2.3.3. Design Pattern

- Design Pattern là một khuôn mẫu, hướng dẫn cách tổ chức các lớp, đối tượng và mối quan hệ giữa chúng để đạt được tính tái sử dụng, dễ bảo trì và mở rộng hệ thống.

- Creational Patterns
 - Singleton
 - Abstract Factory
 - Builder
 - Factory Method
 - Prototype
- Structural Patterns
 - Adapter
 - Bridge
 - Composite
 - Decorator
 - Façade
 - Flyweight
 - Proxy
- Behavioral Patterns
 - Chain of Responsibility
 - Command
 - Interpreter
 - Iterator
 - Mediator
 - Memento
 - Observer
 - State
 - Strategy
 - Template Method
 - Visitor

Hình 2.2. Các nhóm Design Patterns phổ biến

- Các mẫu Design Pattern được sử dụng trong bài:

a. Creational Patterns

- Singleton Pattern – (UserSession):

+ Vị trí: UserSession

+ Mục đích: Đảm bảo chỉ có một phiên người dùng (User Session) duy nhất trong toàn hệ thống.

+ Giải thích: Lớp UserSession có constructor private và thuộc tính tĩnh Instance để truy cập duy nhất, giúp lưu thông tin đăng nhập (username, role, email) trong suốt quá trình chạy mà không bị trùng lặp.

b. Structural Patterns

- Composite Pattern – (IProductComponent, CompositeProduct):

+ Vị trí: IProductComponent, CompositeProduct, Product

+ Mục đích: Cho phép xử lý sản phẩm đơn lẻ và combo sản phẩm giống nhau.

+ Giải thích: CompositeProduct có thể chứa nhiều IProductComponent con, có thể là FoodProduct hoặc DrinkProduct. Khi tính tổng hoặc giảm giá, cả combo được xử lý như một sản phẩm duy nhất.

c. Behavioral Patterns

- Strategy Pattern – (IDiscountStrategy, RegularCustomerDiscountStrategy, VIPCustomerDiscountStrategy)

+ Vị trí: IDiscountStrategy và các lớp kế thừa.

+ Mục đích: Cho phép thay đổi chiến lược giảm giá linh hoạt mà không ảnh hưởng đến lớp Customer.

+ Giải thích: Mỗi loại khách hàng được gán chiến lược giảm giá riêng (Regular giảm 10% tổng giá trị đơn hàng hay VIP giảm 30% tổng giá trị đơn hàng).

2.3.4. Các quan hệ trong Lập trình hướng đối tượng

a. Inheritance (Kế thừa)

- Employee là lớp cha của Manager và Cashier. Giúp chia sẻ các thuộc tính chung như ID, Name,...

- Product là lớp cha trừu tượng (abstract class), được kế thừa bởi FoodProduct, DrinkProduct, ElectronicProduct... Giúp thể hiện đặc trưng của từng sản phẩm, và đảm bảo tính mở rộng khi thêm sản phẩm.

b. Association (Customer - Product)

- Là mối quan hệ một lớp có thể sử dụng hoặc liên kết với một lớp khác, nhưng không sở hữu nó.

+ Biểu diễn mối quan hệ giữa hai lớp độc lập, trong đó một lớp sử dụng hoặc liên kết với lớp khác

+ Mối quan hệ hai chiều hoặc một chiều, không ràng buộc vòng đời.

- Trong đồ án:

+ Customer có danh sách chi tiết đơn hàng (OrderDetails), trong đây nó chứa danh sách sản phẩm của Customer.

c. Aggregation (Order - Product)

- Là mối quan hệ “có – nhưng độc lập”, nghĩa là lớp cha chứa lớp con nhưng vòng đời của chúng tách biệt.

+ Nếu lớp cha bị hủy, lớp con vẫn tồn tại.

- Trong đồ án:

+ Order chứa danh sách các sản phẩm có trong Product. Và khi xóa Order thì các sản phẩm sẽ trả về Product → Được thể hiện bằng sự kiện btnDelete_Click.

d. Composition (Order - OrderDetails)

+ Là mối quan hệ “có – và phụ thuộc hoàn toàn”, vòng đời của lớp con gắn liền với lớp cha.

+ Nếu lớp cha bị hủy, lớp con cũng bị hủy.

- Trong đồ án:

+ Order chứa danh sách OrderDetails. Nếu Order bị xóa, tất cả OrderDetails cũng bị xóa và trả về sản phẩm cho Product → Được thể hiện bằng sự kiện btnDeleteDetail_Click.

+ Invoice chứa InvoiceDetails, không tồn tại độc lập.

e. Dependency

- Là mối quan hệ tạm thời, trong đó một lớp phụ thuộc vào lớp khác để thực hiện một hành động cụ thể, thường thông qua tham số truyền vào phương thức hoặc gọi hàm tạm thời. Nếu lớp được phụ thuộc thay đổi, lớp phụ thuộc có thể phải thay đổi theo.

- Trong đồ án:

+ InvoiceForm phụ thuộc InvoiceData để hiển thị danh sách hóa đơn.

+ UserSession được các Form như MainFormAdmin, MainFormCashier sử dụng để lấy thông tin người đăng nhập.

+ SignIn Form phụ thuộc UserSession.SetUserInfo() khi người dùng đăng nhập.

2.3.5. Kỹ thuật *Serialization* và *Deserialization*

- Serialization là quá trình chuyển đổi các đối tượng trong mã nguồn thành một dạng dữ liệu có thể được lưu trữ hoặc truyền đi qua mạng.

- Để serialize hoặc deserialize một đối tượng trong C#, có nhiều cách triển khai nhưng cần thiết phải thực thi giao diện ISerializable (tất cả các lớp liên quan).

- Ví dụ trong lớp Customer:

[Serializable]

```
public class Customer : ISerializable
{
    // Serialize
    public Customer(SerializationInfo info, StreamingContext context)
    {
        Id = info.GetString("Id");
        Name = info.GetString("Name");
        ...
    }
    // Deserialize
    public void GetObjectData(SerializationInfo info, StreamingContext context)
    {
        info.AddValue("Id", Id);
        info.AddValue("Name", Name);
        ...
    }
}
```

- Sử dụng kỹ thuật bounded generics (các generic có ràng buộc) áp dụng vào BaseDataRepository<TList, TItem> với các ràng buộc:

+ where TList : class, new() — tức là TList phải là class và có constructor không tham số (để có thể new TList() trong Load()/Save()).

+ where TItem : class — tức là TItem phải là reference type.

```
public abstract class BaseDataRepository<TList, TItem>
where TList : class, new()
where TItem : class
```

```

{
    protected readonly string filePath;
    public BaseDataRepository() { }
    protected virtual TList Load() { }
    protected virtual void Save(TList list) { }
    public abstract List<TItem> GetData();
    public abstract void SaveData(List<TItem> items);
    public virtual void CreateSampleData() { }
}

```

- CustomerData kế thừa BaseDataRepository<CustomerList, Customer>:

+ CustomerList thỏa where TList : class, new() (là class và có constructor mặc định).

+ Customer thỏa where TItem : class. => CustomerData sử dụng generic đã được bound.

```

public class CustomerData : BaseDataRepository<CustomerList, Customer>
{
    public CustomerData() : base() { }
    public override List<Customer> GetData() { }
    public override void SaveData(List<Customer> customers) { }
    public override void CreateSampleData() { }
}

```